

CRISP-DM, SEMMA, and KDD Methodologies for Fake News Detection

This document summarizes the CRISP-DM, SEMMA, and KDD data mining methodologies used for the Fake News Detection project. It includes the aim, procedure, result, and expected output.

Aim

To compare the CRISP-DM, SEMMA, and KDD methodologies for fake news detection and identify the most suitable approach for building an accurate machine learning model.

Procedure

CRISP-DM Procedure

- **Business Understanding:** Define the objective of detecting fake news articles.
- **Data Understanding:** Load and explore the dataset.
- **Data Preparation:** Clean the text data, remove stop words, and convert text into numerical features.
- **Modeling:** Train the Logistic Regression classifier.
- **Evaluation:** Measure the model accuracy.
- **Deployment:** Save or use the trained model for prediction.

Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# -----
# CRISP-DM Phase 1: Business Understanding
# Objective: Detect fake news
# -----
```

```
# -----  
  
# CRISP-DM Phase 2: Data Understanding  
  
# -----  
  
df = pd.read_csv("Fake.csv")  
  
  
df["label"] = 0  
  
  
df_true = pd.read_csv("True.csv")  
df_true["label"] = 1  
  
  
df = pd.concat([df, df_true], ignore_index=True)  
  
  
# -----  
  
# CRISP-DM Phase 3: Data Preparation  
  
# -----  
  
X = df["text"]  
y = df["label"]  
  
  
vectorizer = TfidfVectorizer(stop_words="english", max_features=5000)  
X = vectorizer.fit_transform(X)  
  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.2,  
    random_state=42  
)  
  
  
# -----
```

```

# CRISP-DM Phase 4: Modeling
# -----

model = LogisticRegression()

model.fit(X_train, y_train)

# -----

# CRISP-DM Phase 5: Evaluation
# -----

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("CRISP-DM Accuracy:", accuracy)

# -----

# CRISP-DM Phase 6: Deployment
# -----

# import joblib

# joblib.dump(model, "fake_news_model.pkl")

```

SEMMA Procedure

- **Sample:** Load and split the dataset.
- **Explore:** Analyze the news articles.
- **Modify:** Clean text data and apply TF-IDF vectorization.
- **Model:** Train the Logistic Regression classifier.
- **Assess:** Evaluate the model accuracy.

Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score


# -----

# S - Sample

# -----

fake_df = pd.read_csv("Fake.csv")

true_df = pd.read_csv("True.csv")


fake_df["label"] = 0

true_df["label"] = 1


df = pd.concat([fake_df, true_df], ignore_index=True)


# -----

# E - Explore

# -----

X = df["text"]

y = df["label"]


# -----

# M - Modify

# -----

X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y,  
test_size=0.2,  
random_state=42  
)
```

```
vectorizer = TfidfVectorizer(stop_words="english", max_features=5000)
```

```
X_train = vectorizer.fit_transform(X_train)
```

```
X_test = vectorizer.transform(X_test)
```

```
# -----
```

```
# M - Model
```

```
# -----
```

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

```
# -----
```

```
# A - Assess
```

```
# -----
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("SEMMA Accuracy:", accuracy * 100)
```

KDD Procedure

- **Selection:** Load the dataset and select the news content.
- **Preprocessing:** Remove unnecessary text and assign labels.
- **Transformation:** Convert text into TF-IDF features.
- **Data Mining:** Train the Logistic Regression classifier.
- **Interpretation/Evaluation:** Measure and interpret the model accuracy.

Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score


# -----
# KDD Step 1: Selection
# -----

fake_df = pd.read_csv("Fake.csv")
true_df = pd.read_csv("True.csv")

fake_df["label"] = 0
true_df["label"] = 1

df = pd.concat([fake_df, true_df], ignore_index=True)


# -----
# KDD Step 2: Preprocessing
# -----

X = df["text"]
```

```

y = df["label"]

# -----

# KDD Step 3: Transformation

# -----

vectorizer = TfidfVectorizer(stop_words="english", max_features=5000)

X = vectorizer.fit_transform(X)

# -----

# KDD Step 4: Data Mining

# -----

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

model = LogisticRegression()

model.fit(X_train, y_train)

# -----

# KDD Step 5: Interpretation

# -----

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

```

```
print("KDD Accuracy:", accuracy * 100)
```

Result

The Logistic Regression model was trained and evaluated under the CRISP-DM, SEMMA, and KDD methodologies. Since the same dataset, preprocessing steps, train-test split, and model were used, the accuracy obtained is expected to be similar across all three methodologies.

Output

The expected output is the accuracy value printed for each methodology, for example:

CRISP-DM Accuracy: 0.9875

SEMMA Accuracy: 98.75

KDD Accuracy: 98.75

Comparison of Methodologies

Methodology	Main Focus	Strength	Limitation
CRISP-DM	Complete project lifecycle	Comprehensive and industry standard	Requires more documentation
SEMMA	Model building and assessment	Strong analytics workflow	No deployment phase
KDD	Knowledge discovery	Good for research and preprocessing	No deployment phase

Conclusion

Among the three methodologies, **CRISP-DM** is the most suitable for the Fake News Detection project because it covers the complete workflow from business understanding to deployment. SEMMA and KDD are also useful, but they focus on specific parts of the data mining process rather than the entire project lifecycle.